

Module 8 — Core Skills Drill: RAG Basics

This drill is due the evening of the concept session (Sunday). It moves you from "I read about embeddings, Weaviate, and ingestion" to "I can compute an embedding, bring up Weaviate, and ingest a small corpus with externally-supplied vectors." You will not retrieve or evaluate in the drill — that is Monday's lab. The drill targets the operational baseline both the lab and the integration assume.

Due: Tonight, after the concept session. Resubmissions are accepted through Saturday of the assignment week.

Setup

Submission flow change — Drill 8 only (pilot). This drill uses a fork-and-submit flow instead of GitHub Classroom. The lab, integration, and stretch assignments for Module 8 use the regular Classroom flow. Steps 1–3 below are the new part; steps 4–6 are the same as always.

If you haven't already installed the M8 packages and pulled the Weaviate Docker image, go back to the [Setup: install before the concept session](#) section of the reading and complete the install before proceeding. The first-time downloads (Weaviate image ~200 MB, `all-MiniLM-L6-v2` ~80 MB) are large — they were pre-staged on EID-2 (Sun 2026-05-24) so they should already be cached.

- Fork the template repo to your GitHub account.** Open github.com/LevelUp-Applied-AI/m8-d8 and click **Fork** (top right) → set the owner to your personal GitHub account → click **Create fork**. Leave "Copy the main branch only" checked.
- Enable Actions on your fork.** On your forked repo, click the **Actions** tab. GitHub disables workflows on forks by default — click **I understand my workflows, go ahead and enable them**. (One-time per fork. The autograder cannot run on your PR until this is done.)
- Clone your fork (not the upstream repo):**

```
git clone https://github.com/<your-github-username>/m8-d8.git
cd m8-d8
```

- Install dependencies (cached from EID-2 — should be fast):**

```
pip install -r requirements.txt
```

- Start Weaviate (skip if already running from EID-2's smoke test):**

```
docker run -d -p 8080:8080 \
-e PERSISTENCE_DATA_PATH=/var/lib/weaviate \
-e DEFAULT_VECTORIZER_MODULE=none \
-e ENABLE_MODULES= \
--name weaviate \
semitechnologies/weaviate:1.24.10
```

Verify with `curl http://localhost:8080/v1/.well-known/ready` (expect 200).

- Create your working branch:**

```
git checkout -b drill-rag-basics
```

All implementation work goes in `drill_8.py`. The starter file contains function signatures and docstrings — implement each function as described below.

Task 1: Embed a Sentence

Implement `embed_text(text)`:

```
def embed_text(text: str) -> np.ndarray:
    """
    Encode a single string with all-MiniLM-L6-v2 and return the embedding
    as a 384-dimensional float32 NumPy array.

    Returns:
        numpy.ndarray of shape (384,) and dtype float32.
    """
    # TODO: load SentenceTransformer("all-MiniLM-L6-v2") (cached after first call)
    # TODO: encode the text; convert to a float32 numpy array
```

Why this matters: Every dense and hybrid query in the lab will embed the query string before sending it to Weaviate. The drill verifies you can produce a correctly-shaped, correctly-typed embedding — the autograder will catch the common mistakes (returning a torch tensor, returning float64, returning a list of vectors instead of one).

Hint: `SentenceTransformer.encode(...)` returns a NumPy array by default when `convert_to_numpy=True` is passed (the default). The dtype is float32 if you cast it explicitly with `.astype(np.float32)`.

Task 2: Connect and Verify Weaviate

Implement `weaviate_ready(url)`:

```
def weaviate_ready(url: str) -> bool:
    """
    Construct a Weaviate client at the given URL and return whether it is ready.

    A non-running Weaviate (network error, refused connection, etc.) should
    return False — not raise.

    Returns:
        bool: True if the Weaviate instance at `url` responds to `is_ready()`,
            False otherwise.
    """
    # TODO: import weaviate
    # TODO: try to construct weaviate.Client(url) and call .is_ready()
    # TODO: wrap in try/except; return False on any error
```

Why this matters: The lab's autograder runs in CI against a Weaviate service in the workflow YAML. Your local code needs to handle "Weaviate is not up" gracefully — by returning `False`, not by crashing the test run.

Task 3: Ingest a Small Corpus

Implement `ingest_corpus(client, class_name, items)`:

```
def ingest_corpus(client, class_name: str, items: list[dict]) -> int:
    """
    Ingest a small list of items into a Weaviate class.

    Each item is a dict with keys:
        - "title" (str)
        - "text" (str)
        - "vector" (list[float], length 384)

    Define the class schema if it doesn't already exist:
        - vectorizer: "none"
        - properties:
            - title (text)
            - text (text, BM25-indexed)

    Ingest each item with its supplied vector.

    Returns:
        int: the number of objects ingested (verified by querying the
            aggregate count from Weaviate after ingest).
    """
    # TODO: check schema; create class if absent
    # TODO: batch-ingest each item (use client.batch.add_data_object or `with cl
    # TODO: ensure batch is flushed before counting
    # TODO: query aggregate count; return as int
```

Why this matters: Weaviate's batch API is the canonical pattern for ingesting more than a handful of objects. The most common bugs to avoid: forgetting to flush the batch (count comes back as 0), passing the vector as a NumPy array instead of `vector.tolist()` (Weaviate expects a JSON-serializable list), and re-running without deleting the class first (objects accumulate across runs).

Submitting

When all three functions are implemented and pass the local autograder:

- Run the autograder locally:**

```
pytest tests/ -v
```

You will need Weaviate running locally (`docker ps` should show the `weaviate` container). All four tests should pass.

- Commit your work:**

```
git add drill_8.py
git commit -m "Drill 8: embed_text, weaviate_ready, ingest_corpus"
git push origin drill-rag-basics
```

- Open a Pull Request *within your fork*** — base branch `main`, compare branch `drill-rag-basics`. **Important:** the PR must stay inside your fork. When you click "Compare & pull request", GitHub will default to "base repository: LevelUp-Applied-AI/m8-d8" — change the **base repository** dropdown to your own fork (`<your-github-username>/m8-d8`) before opening the PR. The autograder runs against your branch; check the CI status on the PR page to confirm green. If you see "Workflows are disabled" on the Actions tab, you missed Setup step 2 — enable them and push an empty commit (`git commit --allow-empty -m "ci: trigger"`) to re-run.

- Paste your PR URL into TalentLMS → Module 8 → Core Skills Drill to submit this assignment.**

What the Autograder Checks

- `embed_text` returns a NumPy array of shape `(384,)` and dtype `float32`.
- `weaviate_ready` returns `True` for a live Weaviate URL.
- `weaviate_ready` returns `False` (not an exception) for an unreachable URL.
- `ingest_corpus` ingests three test items and returns count `3` (verified by aggregate query).

The autograder runs against a Weaviate service started by GitHub Actions — you do not need to keep your local Weaviate running while CI executes. Make sure your code reads `WEAVIATE_URL` from the environment if set; otherwise default to `http://localhost:8080`.

